Module 8 — Reading: RAG Systems & Vector Retrieval

Everything in this reading is fair game for readiness checks, drills, and labs. A learner who reads this carefully should be able to complete the drill and most of the lab without attending the concept session.

What success looks like: After this reading, you can stand up a local Weaviate vector database; explain when sparse retrieval (BM25), dense retrieval (sentence-transformer embeddings), and hybrid retrieval each succeed and fail; ingest an embedded corpus and query it three different ways; evaluate retrieval with recall@k and Mean Reciprocal Rank (MRR); assemble a Retrieval-Augmented Generation (RAG) pipeline that injects retrieved context into a small generative model; and reason about a coarse content-word-overlap groundedness check on the generator's output.

In This Reading

1. Why Retrieval-Augmented Generation?
2. Setup: install before the concept session
3. Sparse Retrieval — BM25 and the Lexical Baseline
4. Dense Retrieval — Embeddings and the Semantic Baseline
5. Embedding Model Selection
6. Vector Databases — Weaviate
7. Querying Weaviate
8. Retrieval Evaluation — recall@k and MRR
9. RAG End-to-End
10. Generator Evaluation — Groundedness

- Troubleshooting
- Key Terms
- Summary
- What's Next

---

1. Why Retrieval-Augmented Generation?

A pre-trained generative model — even a small one like `flan-t5-base` — has facts baked into its weights from its training data. Ask it about content outside that data and it will guess. The guess is often plausible-sounding and wrong; this is what people mean by "hallucination."

You have two options for fixing this:

1. **Keep retraining the model on new data.** Expensive (compute), slow (training cycles), and only catches updates as fast as your retraining schedule.
2. **Retrieve relevant context at query time and inject it into the prompt.** The model now has the answer in front of it; the only thing it has to do is read and respond from that context. This is **Retrieval-Augmented Generation (RAG)**.

RAG cleanly separates two things:

- **What the model knows in its weights** — fixed at training time, slow to change.
- **What the model needs for *this* query** — fluid, swappable, updateable by changing the corpus rather than retraining.

The user-visible benefits: answers are grounded in a known corpus, citable to specific documents, and updateable by editing the corpus. RAG is the dominant pattern in production question-answering systems for a reason — it solves the freshness and provenance problems that plain generative inference cannot.

The five steps of any RAG system are the same:

1. Accept a query string.
2. Retrieve top-k passages from a corpus.
3. Construct a prompt that injects the retrieved passages as context.
4. Call a generator with that prompt.
5. Return the generator's output.

This week we build all five.

---

2. Setup: install before the concept session

Run these commands before the Sunday concept session. The large Docker image and model downloads should already be cached from the EID-2 install assignment (Sun 2026-05-24); if you skipped EID-2, plan an extra 10–15 minutes for first-time downloads.

Install Python dependencies (drill, lab, integration):

```
pip install -r requirements.txt
```

This installs:

- `sentence-transformers>=2.2,<3` — the embedding library wrapping Hugging Face transformer models.
- `weaviate-client>=3.25,<4` — the Python client for Weaviate; the v3 API is the cohort standard for this module.
- `numpy`, `pandas`, `pytest` — already in your environment from Pre-Work.
- `transformers`, `torch` (CPU build) — already installed from M2 / M6 / M7.

Pull the Weaviate Docker image (pre-pulled on EID-2):

```
docker pull semitechnologies/weaviate:1.24.10
```

Approximately 200 MB. Pre-pulled on EID-2 so first use in the drill is instant.

Pre-download the embedding model (pre-cached on EID-2):

```
python -c "from sentence_transformers import SentenceTransformer; SentenceTransfo
```

Approximately 80 MB. The model is cached to `~/.cache/huggingface/`.

Pre-download the integration generator (pre-cached on EID-2):

```
python -c "from transformers import AutoModelForSeq2SeqLM, AutoTokenizer; AutoMod
```

Approximately 250 MB. Used by the integration task on Wednesday; pre-caching avoids a slow first call when you run the pipeline.

Start a local Weaviate container:

```
docker run -d -p 8080:8080 \
  -e PERSISTENCE_DATA_PATH=/var/lib/weaviate \
  -e DEFAULT_VECTORIZER_MODULE=none \
  -e ENABLE_MODULES= \
  --name weaviate \
  semitechnologies/weaviate:1.24.10
```

`DEFAULT_VECTORIZER_MODULE=none` is load-bearing — it tells Weaviate that we will supply vectors externally (via `sentence-transformers` running in our Python process) rather than asking Weaviate to embed text for us. This keeps the deployment offline-friendly and free.

Healthcheck:

```
curl http://localhost:8080/v1/.well-known/ready
```

A 200 response means Weaviate is up.

Stop / start later:

```
docker stop weaviate     # stop without removing — data persists
docker start weaviate    # restart; volumes preserved
```

The same install carries through to the drill, lab, and integration repos — pip's wheel cache means subsequent `pip install -r requirements.txt` runs are fast (under a minute).

---

3. Sparse Retrieval — BM25 and the Lexical Baseline

The original retrieval mental model: the corpus is a set of documents, each document is a bag of words, and the search engine maintains an **inverted index** mapping every term to the documents it appears in. To answer a query, look up each query term, fetch the document lists, and rank by some scoring function.

BM25 is the dominant scoring function. It rewards a document when it contains query terms that are:

- **Frequent in this document** (term frequency, `tf`) — relevance signal.
- **Rare in the corpus** (inverse document frequency, `idf`) — distinctiveness signal.
- Adjusted for **document length** — short documents that mention a term don't get unfairly out-ranked by long documents that mention it more times.

The exact BM25 formula is a tunable function of `tf`, `idf`, document length, and two parameters (`k1`, `b`). The intuition is more important than the formula:

- Many occurrences of a rare query term → high score.
- Few occurrences of a common query term → low score.
- Long documents are length-normalized so a 10,000-word document that mentions the term twice doesn't beat a 200-word document that mentions it twice.

When BM25 is strong

- **Rare named entities and identifiers.** A query for `error code 0x80070005` will reliably retrieve the document containing that string.
- **Technical terms.** APIs, function names, version strings, error messages.
- **Exact-phrase queries** (when the engine supports phrase matching).

When BM25 is weak

- **Paraphrase.** "How do I fix a memory leak?" and "Debugging heap exhaustion" share almost no terms. BM25 cannot bridge that gap.
- **Synonym mismatch.** "Car" and "automobile" are different tokens.
- **Multilingual queries.** No semantic relationship between tokens in different languages.

BM25 has no training step. It is a deterministic scoring function over an inverted index. This is a feature, not a bug: predictable, fast, no GPU, no model to keep current.

---

4. Dense Retrieval — Embeddings and the Semantic Baseline

You met embeddings in M6 — a fixed-dimensional vector representation of text where semantically similar texts end up at nearby points in the vector space. Dense retrieval makes that operational: embed every document in the corpus once at indexing time; embed each query at search time; rank documents by similarity to the query vector.

DPR — the architectural breakthrough

In 2020 a Facebook AI paper introduced **Dense Passage Retrieval (DPR)**: a **bi-encoder** architecture where the question encoder and the passage encoder are trained jointly so that, for a labeled pair `(question, relevant_passage)`, the dot product `(q, p_relevant)` is larger than `(q, p_irrelevant)` for any other passage. DPR established that, with the right training, dense retrieval can beat BM25 on open-domain question answering.

The "bi-encoder" detail matters: question and passage are encoded **independently** and compared via vector similarity at the end. This means you can embed the entire corpus once, store the vectors, and at query time only do one new embedding (the query). Retrieval becomes "vector similarity search."

`sentence-transformers` — the practical bi-encoder family

DPR's reference implementation requires GPU training and is awkward to run on CPU. The `sentence-transformers` Python library provides a CPU-runnable family of bi-encoder models — same conceptual architecture, different model weights, much easier to run. We use `sentence-transformers` for the operational work in this module.

```
from sentence_transformers import SentenceTransformer
model = SentenceTransformer("all-MiniLM-L6-v2")
vectors = model.encode(["how do I free memory in Python?", "garbage collection in
# vectors is a (2, 384) numpy array
```

Cosine similarity vs. dot product

The two standard similarity functions:

- **Cosine similarity:** `(a · b) / (||a|| ||b||)`, in `[-1, 1]`.
- **Dot product:** `a · b`, unbounded.

When vectors are L2-normalized (length 1), cosine and dot product are identical. `sentence-transformers` MiniLM and mpnet models output normalized vectors by default. Weaviate's default distance for the HNSW index is `cosine`. Be explicit about both — if your model produces unnormalized vectors and your index uses cosine, results are still correct; if your index uses dot product but your vectors aren't normalized, large vectors win unfairly.

When dense retrieval is strong

- **Paraphrase.** "fix memory leak" and "debug heap exhaustion" land at nearby points if both occur with similar context in the model's training data.
- **Semantic similarity across vocabulary.**
- **Cross-domain transfer** — a model trained broadly handles new domains better than handcrafted lexical rules.

When dense retrieval is weak

- **Rare entities not seen in training.** A novel error code or product SKU may have no good vector representation; the model essentially guesses.
- **Queries dominated by exact identifiers.** `error code 0x80070005` — BM25 finds it; dense retrieval may not.
- **Domain shift far from the training corpus.** Specialized jargon (legal, clinical) where the model's training data is thin.

---

5. Embedding Model Selection

`sentence-transformers` ships a catalog of pre-trained models. Two are most relevant for this module:

Model	Dimensions	Size	Speed (CPU)	Quality
`all-MiniLM-L6-v2`	384	~80 MB	Fast	Good
`all-mpnet-base-v2`	768	~420 MB	~3× slower	Better

Cohort default: `all-MiniLM-L6-v2`. Fast, small, and good enough that you will not notice the gap to mpnet on a 1,000-document corpus.

A common reflex is "larger model is always better." For embeddings, the relationship saturates: doubling dimensions roughly doubles indexing storage, increases query latency, and only marginally improves recall on most corpora. The right move is to start with MiniLM, profile latency and recall@5, and only move to mpnet if you measure a meaningful lift on your evaluation set.

---

6. Vector Databases — Weaviate

You could store embeddings in a NumPy array, run cosine similarity by hand, and call it a vector database. For 1,000 documents on your laptop, that even works. For the back half of this program — and for any production system — you want something more capable:

- **Persistence** — the index survives restarts.
- **Approximate Nearest Neighbor (ANN) indexes** — sublinear query time. Brute-force cosine over 1M vectors is slow; ANN indexes (HNSW, IVF, ScaNN) get you near-perfect recall in milliseconds.
- **Hybrid query support** — combine vector similarity with keyword filters in one query.
- **Schema-aware filtering** — `vector_search WHERE category = 'auth'`.
- **The operational story** — deploy, scale, monitor, back up.

Weaviate is one of several open-source vector databases (others include Qdrant, Milvus, pgvector). We chose Weaviate for this course because:

- It is open-source and CPU-runnable in a single Docker container.
- It supports BM25, vector, and hybrid retrieval out of the box — exactly the three modes we want to compare.
- The Python client (`weaviate-client`) has a clean, learnable API.

HNSW conceptually

The default Weaviate index is **Hierarchical Navigable Small World (HNSW)** — a graph-based ANN structure where each vector becomes a node, edges connect nearby vectors at multiple zoom levels, and queries navigate the graph from coarse to fine. The result is sub-linear query time with near-perfect recall in practice. You will not implement HNSW; you will use it through the Weaviate API.

Weaviate's data model

A **class** (sometimes called a "collection" in newer Weaviate documentation) is a typed group of objects with a shared schema. Each object in a class has:

- A set of **properties** — typed fields like `text`, `int`, `boolean`, `date`. Properties can be marked filterable (for `WHERE`-style queries) and BM25-indexed (for keyword search).
- A **vector** — one per object. Either supplied by you (`vectorizer: "none"`) or computed by Weaviate via a configured module.

This module always uses `vectorizer: "none"` and supplies vectors from `sentence-transformers` running in our Python process. This decision is intentional:

- **No third-party API calls from Weaviate.** No accounts, no API keys, no rate limits.
- **You control which model produces the vectors.** Swapping models is just a matter of re-running the embedder; Weaviate doesn't need reconfiguring (beyond recreating the class if dimensions change).
- **Free.** No paid embedding service in the loop.

A minimal class definition (Python, v3 client):

```
client.schema.create_class({
    "class": "Post",
    "vectorizer": "none",
    "vectorIndexConfig": {"distance": "cosine"},
    "properties": [
        {"name": "doc_id", "dataType": ["text"]},
        {"name": "title", "dataType": ["text"]},
        {"name": "answer_text", "dataType": ["text"]},
    ],
})
```

> *The lab schema extends this minimal example with `subset`, `question_text`, and `text`, and adds the per-property `indexSearchable` / `indexFilterable` / `tokenization` knobs that decide which properties BM25 actually scores against. See the lab guide for the exact mapping.*

---

7. Querying Weaviate

The v3 Python client query shape is consistent across modes — start with `client.query.get(<class>, [<properties>])`, chain a retrieval-mode method, set a limit, and call `.do()`.

Three retrieval modes

Pure dense (vector similarity):

```
qv = embedder.encode("how do I free memory?").tolist()
result = (
    client.query
    .get("Post", ["doc_id", "title"])
    .with_near_vector({"vector": qv})
    .with_limit(10)
    .do()
)
```

Pure sparse (BM25):

```
result = (
    client.query
    .get("Post", ["doc_id", "title"])
    .with_bm25(query="how do I free memory?")
    .with_limit(10)
    .do()
)
```

Hybrid (BM25 + vector with `alpha` weighting):

```
qv = embedder.encode("how do I free memory?").tolist()
result = (
    client.query
    .get("Post", ["doc_id", "title"])
    .with_hybrid(query="how do I free memory?", vector=qv, alpha=0.5)
    .with_limit(10)
    .do()
)
```

`alpha = 1.0` weights toward pure vector; `alpha = 0.0` weights toward pure BM25; `alpha = 0.5` is balanced.

Filtering by property

You can constrain retrieval to a subset of the corpus by chaining `.with_where(...)`:

```
result = (
    client.query
    .get("Post", ["doc_id", "title"])
    .with_near_vector({"vector": qv})
    .with_where({"path": ["subset"], "operator": "Equal", "valueText": "android"}
    .with_limit(10)
    .do()
)
```

Filters compose with any retrieval mode.

Reading the result

The `result` is a nested dict shaped like `{"data": {"Get": {"Post": [{"doc_id": "...", "title": "..."}, ...]}}}`. Practical code extracts the inner list:

```
hits = result["data"]["Get"]["Post"]
ids = [hit["doc_id"] for hit in hits]
```

A note about `alpha = 0` and `alpha = 1` in v3

The v3 client's hybrid implementation has historically had edge-case behavior at the extreme alpha values. For predictability — especially in autograded tests — do **not** use `alpha=0.0` or `alpha=1.0` to mean "pure BM25" or "pure vector." Call `with_bm25(...)` or `with_near_vector(...)` directly when you want a pure mode, and reserve hybrid for genuine blending (e.g., `alpha=0.5`).

---

8. Retrieval Evaluation — recall@k and MRR

Through M7 you have known evaluation as a single-number-per-prediction story: accuracy, F1, ROUGE. Retrieval is different — you produce a **ranked list** and the metric needs to know how good the *list* is. Two metrics dominate.

The labeled set

Both metrics need a labeled list of `(query, gold_doc_id)` pairs — for each query, which document is the right answer. You build this either by (a) human annotation or (b) using existing labeled data (this module's lab uses CQADupStack qrels — real Stack Exchange duplicate-question relations — for paraphrastic queries, and hand-authored factoid queries for the rest).

recall@k

The fraction of queries where the gold document appears anywhere in the top-k retrieved results.

```
recall@k = (count of queries where gold_doc_id ∈ top_k) / total_queries
```

- `recall@1` measures pinpoint accuracy — was the gold doc the first result?
- `recall@5` and `recall@10` measure "is the right answer in the candidate set we'll pass to the generator?"

For a RAG pipeline that retrieves k=5 contexts to feed the generator, `recall@5` is the *ceiling* on the system's correctness: if the right document didn't make it into the top 5, the generator can't possibly find the answer there.

MRR (Mean Reciprocal Rank)

For each query, find the 1-indexed position (rank) of the gold document in the retrieved list. Compute `1 / rank`. If the gold document is not in the top-k at all, contribute 0. Average across queries.

```
MRR = (1/N) · Σ (1 / rank_i)   where rank_i = ∞ contributes 0
```

- Gold doc at position 1 contributes 1.0.
- Gold doc at position 2 contributes 0.5.
- Gold doc at position 5 contributes 0.2.
- Gold doc not found contributes 0.

MRR rewards getting the right answer **highly ranked**, not just present. A retriever with `recall@5 = 1.0` and `MRR = 0.2` puts the gold doc somewhere in the top 5 but rarely first — that's a re-ranking opportunity (Thursday's stretch).

Why both

- **`recall@5`** answers: *will the generator have the right context?*
- **`MRR`** answers: *is the retrieval ordering meaningful?*

Reporting both exposes the "right context, wrong order" failure mode that a single number would hide.

---

9. RAG End-to-End

The five-step pattern from §1 in code form:

```
def rag_pipeline(query: str, k: int = 5) -> dict:
    contexts = retrieve(query, k)              # step 2: top-k passages
    prompt = build_prompt(query, contexts)     # step 3: context-injected prompt
    answer = generate(prompt)                  # step 4: call generator
    return {"query": query, "answer": answer, "contexts": contexts, "prompt": pro
```

Choosing top-k

Too small → the generator might miss the relevant passage. Too large → context overflows the model's input window and the signal-to-noise ratio drops. **Cohort default: k = 5.** For a corpus of ~1,000 documents and `flan-t5-base`'s 512-token input window, k=5 is the sweet spot.

Prompt template

A worked example for `flan-t5-base`:

```
Answer the question using only the context. If the context does not contain the a

Context:
[1] {title_1}: {answer_text_1}
[2] {title_2}: {answer_text_2}
[3] {title_3}: {answer_text_3}
[4] {title_4}: {answer_text_4}
[5] {title_5}: {answer_text_5}

Question: {query}
Answer:
```

A few design choices to notice:

- **"Answer only from the context"** — the abstention instruction. Tells the model to ground its response in the supplied passages and refuse if the context doesn't contain the answer. Smaller instruction-tuned models follow this more conservatively than larger ones (often abstaining even when the answer *is* present); larger models follow it more loosely. The integration task makes this trade-off observable.
- **Numbered `[i]:` markers** — give the model orienting cues and create a citation-like structure. If you later want the model to emit `[2]`-style citations (Tier 2 challenge), the framework is already in place.
- **`{title_i}: {answer_text_i}`** — the per-context substitution uses the *answer* portion of the retrieved post, truncated to the first 80 whitespace-split tokens. Why answer-only and not the full document? Because the question portion of each retrieved post is largely redundant with the learner's query (both are questions about the same topic) and consumes context budget that should go to the answer content. This is a load-bearing structural decision specific to question-answer corpora — for other corpora you'd structure context selection differently.
- **Truncation** — `flan-t5-base`'s input window is ~512 tokens. Five passages × 80 tokens + the prompt scaffolding fits comfortably; longer passages would silently truncate at the tokenizer.

Calling the generator

`flan-t5-base` is a sequence-to-sequence model. The standard call:

```
from transformers import AutoModelForSeq2SeqLM, AutoTokenizer

# Load once at module level — not per call
tokenizer = AutoTokenizer.from_pretrained("google/flan-t5-base")
model = AutoModelForSeq2SeqLM.from_pretrained("google/flan-t5-base")

def generate(prompt: str) -> str:
    inputs = tokenizer(prompt, return_tensors="pt", truncation=True, max_length=5
    outputs = model.generate(**inputs, max_new_tokens=128, num_beams=1)
    return tokenizer.decode(outputs[0], skip_special_tokens=True)
```

Two important defaults:

- **`num_beams=1`** (greedy decoding) — reproducible across runs on CPU. Beam search produces slightly better outputs but adds nondeterminism unless seeded carefully.
- **Loading the model once at module level** — the `from_pretrained` call takes several seconds. Loading inside `generate()` per call makes evaluation 40× slower. (TAs will flag this in code review even if the autograder doesn't fail on it.)

---

10. Generator Evaluation — Groundedness

The professional definition of a good RAG output: every claim in the answer is supported by the retrieved context. A grounded answer doesn't have to be *correct* in some absolute sense — it has to be **traceable to the context**. (If the context is wrong, that's a retrieval problem or a corpus problem, not a generation problem.)

Hand-grading groundedness across hundreds of (answer, context) pairs is expensive. Production systems use stronger automated signals (NLI models, model-as-judge, citation validation) — those are deferred to capstone-level work. For this module we use a coarser proxy that is cheap, transparent, and good enough to catch obvious failures.

Content-word overlap

Compute the proportion of the answer's content words that also appear in the concatenated context.

1. Lowercase both `answer` and concatenated `contexts[*].answer_text` (the same content shown to the generator in the prompt).
2. Tokenize on whitespace + punctuation.
3. Remove stopwords using the hardcoded ~50-word English stopword list shipped in the integration starter (`STOPWORDS` constant in `rag_service.py`). No external library — the cohort env doesn't include NLTK and we don't add it for this single use.
4. Compute `|answer_tokens ∩ context_tokens| / |answer_tokens|`. (A Jaccard-style intersection-over-answer ratio, not over the union — we're asking "how much of the answer was sourced from the context," not "how much do they overlap symmetrically.")
5. **Empty-answer edge case:** return 0.0 if the answer is empty after stopword removal.

The score is in `[0, 1]`:

- **≥ 0.6** — likely grounded. Most of the answer's content words came from the context.
- **0.4 – 0.6** — mixed. Worth inspecting.
- **< 0.4** — many tokens not in the context. Possible hallucination, possible aggressive paraphrase, possibly the model is producing a generic answer.

What this metric catches well

- **Flat-out hallucination.** A claim in the answer that has no analog in the context — score drops sharply.
- **Off-topic generations.** A model that produces "I don't have access to the internet" when shown a context — score near 0.
- **Stylistic noise.** Boilerplate that doesn't reference the context.

What this metric misses

- **Paraphrase that *is* grounded.** "The patient is allergic to penicillin" → "Avoid penicillin in this patient." Same fact, different words, low overlap.
- **Arithmetic.** "The article says 5%, and 5% of 200 is 10." The "10" isn't in the context.
- **Negation.** "The drug should NOT be used in pregnancy" overlaps fully with "The drug should be used in pregnancy" by content-word count.

A 0.5 score is a *flag*, not a verdict. Inspect the answer; the metric is doing its job by drawing your attention.

Troubleshooting

- **Weaviate not reachable.** Check `docker ps` to verify the container is running and `curl http://localhost:8080/v1/.well-known/ready` returns 200. If you remapped the port, update `weaviate.Client(url="http://localhost:<port>")`.
- **`vectorizer: none` but no vector supplied at ingest.** Weaviate accepts the object with a null vector — but vector queries return nothing for that object. Always pass `vector=...` in `client.batch.add_data_object(...)`.
- **Embedding-model dimensionality mismatch.** If you ingest with MiniLM (384-dim) and later query with mpnet (768-dim), Weaviate errors. The class schema's vector dimensions are fixed at class creation; switching models requires `client.schema.delete_class("Post")` and re-creating.
- **Cosine vs. dot product mismatch.** `sentence-transformers` MiniLM/mpnet produce normalized vectors and Weaviate's default distance is `cosine` — they line up. Be explicit in the schema (`"vectorIndexConfig": {"distance": "cosine"}`) so future you doesn't get confused.
- **`with_bm25(...)` returns nothing.** BM25 only searches properties indexed for BM25 (configured at class creation via tokenization settings). If your `text` property isn't BM25-indexed, sparse queries will return empty results.
- **Top-k larger than corpus size.** Weaviate returns however many it has, no error. Don't assert exact list length in tests unless you know the corpus size.
- **`flan-t5-base` truncating long contexts.** Input limit is ~512 tokens. Truncate each passage to ~80 tokens (the cohort default); reduce k if you need more headroom per passage.
- **First model load is slow.** `SentenceTransformer(...)` and `AutoModelForSeq2SeqLM.from_pretrained(...)` download weights on first use. Pre-download via the EID-2 install assignment.
- **Re-running the lab leaves stale Weaviate state.** Schemas and ingested objects persist across `docker stop / docker start`. Call `client.schema.delete_class("Post")` at the top of `index_corpus` to start fresh, or use a fresh Docker volume.
- **Groundedness score of 0.5 looks "ungrounded" but the answer is correct.** Content-word overlap misses paraphrase. Inspect the answer — the metric is a flag, not a verdict.

Key Terms

- **alpha (in hybrid retrieval)** — weighting parameter in `[0, 1]`; higher = more vector contribution, lower = more BM25.
- **ANN (Approximate Nearest Neighbor)** — a family of algorithms for finding near-nearest vectors quickly with small recall loss.
- **BM25** — a sparse retrieval scoring function based on term frequency, inverse document frequency, and document-length normalization.
- **bi-encoder** — a dual-tower architecture where query and document are encoded independently and compared via vector similarity.
- **class / collection (Weaviate)** — a typed group of objects with shared schema and a single vector index.
- **content-word overlap** — non-stopword token overlap between two strings, used as a coarse groundedness proxy.
- **cosine similarity** — `(a · b) / (||a|| ||b||)`; a similarity metric in `[-1, 1]` for non-zero vectors.
- **DPR (Dense Passage Retrieval)** — a 2020 bi-encoder retrieval architecture from Facebook AI that established dense retrieval competitiveness on open-domain question answering.
- **embedding** — a fixed-dimensional vector representation of text.
- **groundedness** — the property that an answer's claims are supported by the retrieved context.
- **HNSW (Hierarchical Navigable Small World)** — a graph-based ANN index used as the default in Weaviate.
- **hybrid retrieval** — a retrieval strategy that combines BM25 and dense scores into a single ranking.
- **inverted index** — a mapping from terms to the documents they appear in; the data structure underlying BM25.
- **MRR (Mean Reciprocal Rank)** — the average of `1/rank_of_gold_doc` across queries; rewards getting the right answer highly ranked.
- **prompt template** — a string with placeholders that is filled in with the query and retrieved context before being sent to the generator.
- **RAG (Retrieval-Augmented Generation)** — a pattern that retrieves relevant context and injects it into a generator's prompt.
- **recall@k** — the fraction of queries where the gold document appears in the top-k retrieved.
- **`sentence-transformers`** — a Python library wrapping Hugging Face transformer models trained for sentence-level embedding.
- **sparse retrieval** — retrieval based on term-frequency representations (BM25, TF-IDF).
- **top-k** — the number of retrieved passages returned for a query.
- **vector database** — a database optimized for storing and querying high-dimensional vectors with ANN indexes.
- **`vectorizer: "none"`** — a Weaviate class configuration where vectors are supplied by the client rather than computed by Weaviate.

Summary

RAG separates "what the model knows" from "what the model retrieves." Building a RAG system is five steps: query, retrieve, build prompt, generate, return.

Retrieval comes in three modes: **BM25** (sparse, lexical, strong on rare entities and exact identifiers), **dense** (embedding-based, strong on paraphrase and semantic similarity), and **hybrid** (a weighted blend of the two). Weaviate, a CPU-runnable vector database, supports all three through a uniform Python API. We supply embeddings externally (`vectorizer: "none"`) using `sentence-transformers` — the cohort default model is `all-MiniLM-L6-v2` at 384 dimensions.

Retrieval is evaluated with **recall@k** (will the gold doc make it into the candidate set?) and **MRR** (is the right answer ranked highly?). Both numbers report different aspects of retrieval quality and should be tracked together.

The generation stage uses `flan-t5-base` — a small, CPU-runnable, instruction-tuned model. We feed it a prompt template that includes the question and the retrieved context, and instruct it to "answer only from the context." Generator output is evaluated with a coarse **content-word overlap** groundedness check — useful for catching flat-out hallucination, less useful at distinguishing grounded paraphrase from hallucination.

This week's pattern — retrieve, evaluate retrieval, generate, evaluate generation — is the architectural template for RAG systems in production.

What's Next

- **Sunday evening (Drill).** Stand up Weaviate locally, embed a sentence with `sentence-transformers`, ingest a small corpus with externally-supplied vectors. Mechanical proof you can run the rails.
- **Monday (Lab — Applied Lab).** Build the retrieval layer end-to-end on the Stack Exchange Q&A corpus: schema, ingest, BM25 / dense / hybrid retrievers, recall@5 / recall@10 / MRR, and a comparison brief.
- **Wednesday (Integration Task).** Add the generator. Implement `retrieve → build_prompt → generate → groundedness_score → evaluate_rag`. Encounter and articulate the asymmetric story between retrieval correctness and generator behavior.
- **Tuesday (Stretch — Honors Track).** Query Router — a routing layer that classifies queries and dispatches to the best retrieval mode.
- **Thursday (Stretch — Honors Track).** Cross-Encoder Re-Ranking — a two-stage retriever that adds a re-ranker on top of hybrid retrieval.

Module 9 layers structured retrieval (knowledge graphs, SPARQL) on top of the vector retrieval established here. Modules 10–11 wrap the M8 RAG pipeline behind a FastAPI service and add monitoring around retrieval latency and groundedness. The capstone often combines the M8 RAG service with the M9 KG service into an end-to-end question-answering system.

© 2026 LevelUp Economy. All rights reserved. Unauthorized reproduction or distribution of this material is prohibited.